HDOC DAY – 55

ALGORITHM:

Algorithm: Sliding Window Maximum

1. Start.
2. Read the size of the array size.
3. Declare the variable-length array arr[size].
4. Read all elements of arr.
5. Read the window size k.
6. Pass the address of arr to window_maximum().
7. In window_maximum():
    - If k <= 0 or k > size, display -1 and return.
    - Use an outer loop from i = 0 to size - k.
    - For each window:
        - Set max equal to the first element of the window.
        - Use an inner loop to check the remaining k-1 elements.
        - If an element is greater than max, update max.
        - Display max.
8. Repeat until all windows are processed.
9. Stop.

C PROGRAM:

```
#include <stdio.h>


void window_maximum(int *arr, int size, int k)
{
   int i, j, max;
```

```
// Check if k is invalid
if (k <= 0 || k > size)
{
   printf("-1\n");
   return;
}

// Move the sliding window from left to right
for (i = 0; i <= size - k; i++)
{
   // Assume first element of current window is maximum
   max = *(arr + i);

   // Check all elements in the current window
   for (j = i + 1; j < i + k; j++)
   {
      if (*(arr + j) > max)
      {
         max = *(arr + j);
      }
   }

   printf("%d", max);
```

```
        // Print space except after the last maximum
        if (i < size - k)
            printf(" ");
    }

    printf("\n");
}

int main()
{
    int size, k, i;

    printf("Enter the size of array: ");
    scanf("%d", &size);

    int arr[size];   // Variable Length Array

    printf("Enter the array elements: ");
    for (i = 0; i < size; i++)
    {
        scanf("%d", &arr[i]);
    }

    printf("Enter k: ");
    scanf("%d", &k);
```

```
    // Passing address of array
    window_maximum(arr, size, k);


    return 0;
}
```

TEST CASE TABLE:

Test Case	Array	k	Expected Output
1	1 3 -1 -3 5 3 6 7	3	3 3 5 5 6 7
2	1 2 3 4 5	2	2 3 4 5
3	1 2 3	4	-1
4	5 4 3 2 1	2	5 4 3 2
5	2 7 1 8 3	1	2 7 1 8 3